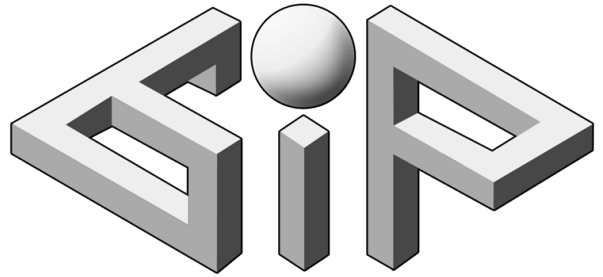
Learning Eigenstructures of Unstructured Data Manifolds

Roy Velich[1] Arkadi Piven[1] David Bensaïd[1] Daniel Cremers[2,3]
Thomas Dagès[1,2,3] Ron Kimmel[1]

In memory of Haïm Brezis (1944 – 2024).



[1]Technion - Israel Institute of Technology
[2]Technical University of Munich
[3]Munich Center for Machine Learning

Mathematical Background

The Setup — The Domain

A domain $\mathcal{M}$ is "where our signals live“.

Domain Type	Example	Structure
Regular	Interval $[0,1]$, 2D grid	Uniform, periodic
Surface	Mesh of a 3D shape	Irregular connectivity
Point cloud	3D scan, image embeddings	No explicit connectivity
Graph	Social network, molecules	Discrete nodes + edges

The Setup — Signals

A signal is a function $f: \mathcal{M} \to \mathbb{R}$.

Domain	Signal Example
Interval	Audio waveform
2D grid	Grayscale image
3D surface	Heat distribution, texture
Image manifold	Any scalar property of images

Discrete version: If $\mathcal{M}$ has N points, then $f \in \mathbb{R}^N$

The Setup — Family of Signals

A set $\mathcal{C}$ of signals that share common structure.

Family $\mathcal{C}$	Shared Property
Natural images	Spatially smooth, edges are sparse
Temperature maps	Vary gradually, no random jumps
Audio signals	Bandlimited, harmonic structure
Shape descriptors	Smooth along surface

Question: **How do we describe a family mathematically?**

The Setup — Family of Signals

We define a family of signals via a constraint operator L:

$$\mathcal{C}_L = \{f : \|f\|_L \leq 1\}$$

Where $\|f\|_L^2 = \langle f, Lf \rangle$, and L is symmetric positive-definite operator.

The operator L encodes our prior:

- Large $\|f\|_L$, signal violates our assumptions (bad)
- Small $\|f\|_L$, signal conforms to our prior (good).

The Fundamental Problem — Signal Approximation

Goal: Approximate signals from $\mathcal{C}_L$ using k basis functions.

Given an orthonormal basis $b = (b_1, \dots, b_n)$ the k-term approximation of f is:

$$\hat{f}_k = \sum_{i=1}^{k} \langle f, b_i \rangle b_i \approx f$$

The reconstruction error:

$$\left\| f - \hat{f}_k \right\|$$

The question:

- Which orthonormal basis b gives the best approximation for signals in $\mathcal{C}_L$?
- What does "best" mean?

Two Approaches to "Best"

- Approach 1: Min-Max (Worst-Case)
- Approach 2: PCA (Average-Case)

Approach 1: Min-Max (Worst-Case)

Minimize the worst reconstruction error over all signals in $\mathcal{C}_L$:

$$\alpha_k(b) = \max_{f \in \mathcal{C}_L} \left\| f - \sum_{i=1}^{k} \langle f, b_i \rangle b_i \right\|$$

Find b that minimizes $\alpha_k(b)$.

$$b = \operatorname*{argmin}_{b \in \mathcal{B}} \max_{f \in \mathcal{C}_L} \left\| f - \sum_{i=1}^{k} \langle f, b_i \rangle b_i \right\|$$

***Intuition**: No signal in our class is poorly approximated*.

Approach 2: PCA (Average-Case)

Minimize expected reconstruction error:

$$\mathbb{E}_{f\sim\mathcal{C}_L}\left[\left\|f-\sum_{i=1}^{k}\langle f,b_i\rangle b_i\right\|^2\right]$$

This is equivalent to maximizing the captured variance:

$$\mathbb{E}_{f\sim\mathcal{C}_L}\left[\sum_{i=1}^{k}\langle f,b_i\rangle^2\right]=\mathbb{E}_{f\sim\mathcal{C}_L}\left[\|\hat{f}_k\|^2\right]$$

***Intuition**: On average, signals are well approximated*.

Approach 2: Why Maximizing Variance ⟺ Minimizing Reconstruction Error

Projection creates an orthogonal decomposition:

$$f = \underbrace{\hat{f}_k}_{\text{projection}} + \underbrace{(f - \hat{f}_k)}_{\text{residual}}$$

Pythagorean theorem:

$$\|f\|^2 = \left\|\hat{f}_k\right\|^2 + \left\|f - \hat{f}_k\right\|^2$$

Taking expectations:

$$\underbrace{\mathbb{E}[\|f\|^2]}_{\textit{total variance (fixed)}} = \underbrace{\mathbb{E}\left[\left\|\hat{f}_k\right\|^2\right]}_{\textit{captured variance}} + \underbrace{\mathbb{E}\left[\left\|f - \hat{f}_k\right\|^2\right]}_{\textit{reconstruction error}}$$

Since total variance is fixed:

Max captured variance ⟺ Min reconstruction error

Approach 1 ⟺ Approach 2

Theorem (Brezis et al., 2016):

Both approaches yield the **same** solution:

	Min-Max	**PCA**
Optimal basis	Eigenvectors of L	Eigenvectors of L
Order	Ascending λ_i	Ascending λ_i

Additionally, The worst-case error using eigenvectors $e_1, \dots, e_k$ equals $1/\lambda_{k+1}$:

$$\alpha_k = \frac{1}{\lambda_{k+1}}$$

Smoothness as a Natural Prior

For most natural signals, the prior is smoothness:

- Neighboring values are similar
- No random jumps or wild oscillations

Smoothness means bounded ***Dirichlet energy***:

$$\int_{\mathcal{M}} \|\nabla f\|^2 dA = \|f\|_{\Delta}^2 = \langle f, \Delta f \rangle$$

Therefore, a natural prior is to set $L = \Delta$ (Laplacian). The optimal basis for smooth signals is the Laplacian eigenbasis!

Summary — Putting It Together

The operator we choose: Laplacian

The signal class: smooth functions

$$\mathcal{C}_\Delta = \{f : \| f \|_\Delta \leq 1\}$$

The goal: recover Laplacian eigen-decomposition via PCA

- The optimal basis for $\mathcal{C}_\Delta$ is the Laplacian eigen-basis.
- Minimizing reconstruction error over smooth functions recovers eigenvectors of Δ.
- Eigenvalues come from reconstruction errors: $\lambda_{k+1} = 1/\alpha_k$.
- We don't need to construct Δ explicitly — just sample smooth functions and minimize reconstruction error!

Why the Laplacian's Eigen-Decomposition is Useful in Practice?

Shape Analysis & Geometry Processing

- Shape descriptors (Heat Kernel Signature, Wave Kernel Signature)
- Mesh smoothing and denoising
- Mesh parameterization and segmentation
- Shape interpolation and morphing

Functional Maps & Correspondence

- Compact representation for shape matching
- Maps encoded as small matrices in spectral domain

Manifold Learning & Dimensionality Reduction

- Laplacian Eigenmaps
- Diffusion Maps

Signal Processing on Manifolds

- Filtering (low-pass, band-pass)
- Compression
- Denoising

Physics Simulation

- Heat diffusion
- Wave propagation

Traditional Pipeline and Related Works

Traditional Pipeline — Computing the Laplacian and its Eigen-Decomposition

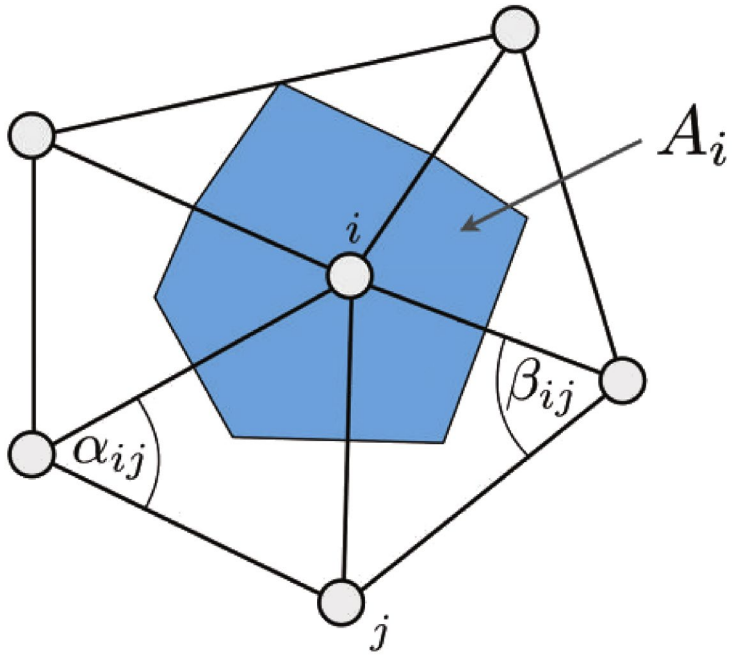
- **Step 1**: Build the Stiffness Matrix $S \in \mathbb{R}^{N \times N}$
- **Step 2**: Build the Mass Matrix $M \in \mathbb{R}^{N \times N}$
- **Step 3**: Form the Laplacian:
  - Unnormalized version: $L = M^{-1}S$
  - Symmetric normalized version: $L_{sym} = M^{-1/2}SM^{-1/2}$
- **Step 4**: Solve Generalized Eigenvalue Problem
  - $S\phi = \lambda M\phi$
  - Use sparse eigensolvers (Lanczos, ARPACK, etc.)

Mesh $\rightarrow$ ***Build*** $\boldsymbol{S, M}$ $\rightarrow$ ***Eigensolver*** $\rightarrow$ ***Eigenbasis***

The Cotangent Laplacian

$$S_{ij} = \begin{cases} -\frac{1}{2}\left(\cot\alpha_{ij} + \cot\beta_{ij}\right) & i \neq j,\ adjacent \\ -\sum_{k \neq i} S_{ik} & i = j \\ 0 & otherwise \end{cases}$$

$$M_{ii} = A_i = \frac{1}{3} \sum_{t \in \text{Triangles}} \text{Area}(t)$$



https://rodolphe-vaillant.fr/entry/101/definition-laplacian-matrix-for-triangle-meshes

A Laplacian for Nonmanifold Triangle Meshes (Sharp et al. 2020)

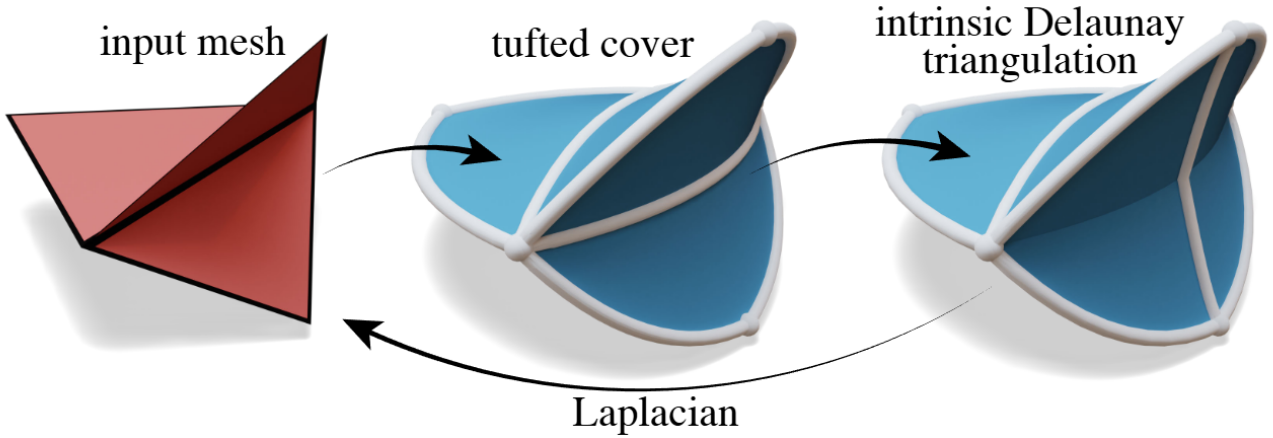


Figure 8: *Starting with the input mesh, we build a tufted cover, then flip to intrinsic Delaunay. Since the vertex set is preserved, a Laplacian built on the iDT can be used directly on the input mesh.*

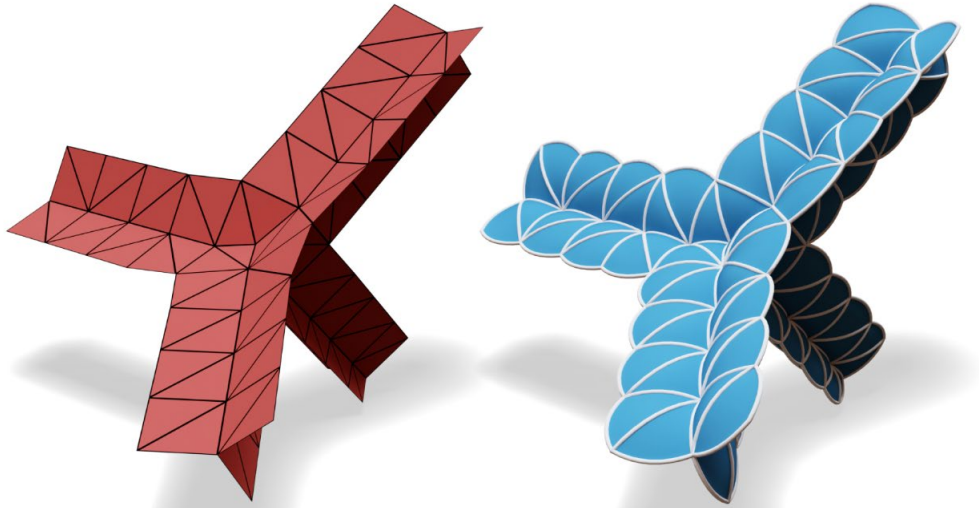
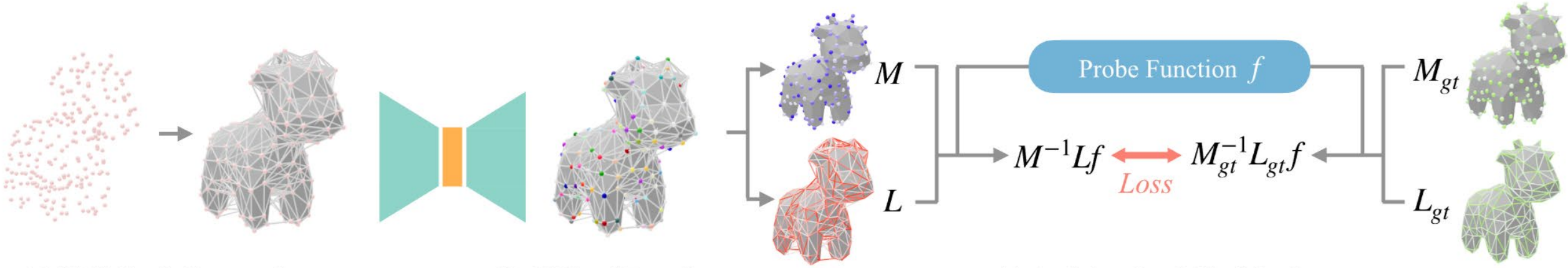


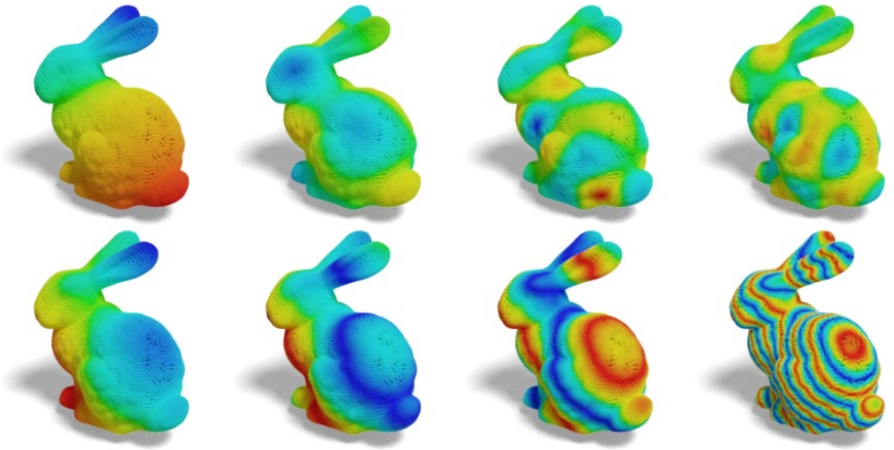
Figure 2: *A nonmanifold mesh (*left*) and its tufted cover (*right*). Since the cover is edge-manifold, we can freely flip edges in order to improve the Laplacian. (Note that we draw curved triangles only to help visualize connectivity; actual triangles remain flat.)*

https://www.cs.cmu.edu/~kmcrane/Projects/NonmanifoldLaplace/NonmanifoldLaplace.pdf

Neural Laplacian Operator for 3D Point Clouds (Pang et al. 2024)



(a) KNN Graph Construction (b) GNN and Decoding (c) Applying f and Obtaining Loss



What is Common for all of Them?

- All methods require to assemble the stiffness and mass matrices.
- All methods use spare eigensolvers.
- **All methods assume an intrinsic dimension of 2 (2-manifold).**

Our Approach

Our Idea: Exploit Brezis' Theorem to Directly Learn the Eigen-Decomposition

Traditional

Mesh → *Build* S, M → *Eigensolver* → *Eigenbasis*

Ours

Point *Cloud* → *Neural Network* → *Eigenbasis*

We train a network to predict eigenfunctions by minimizing reconstruction error on smooth probe functions.

What We Bypass:

- No mesh connectivity
- No explicit operator construction
- No numerical eigensolver

What We Gain:

- Works on raw point clouds
- Agnostic to intrinsic dimension

Two Variants of the Laplacian

Unnormalized Laplacian

$$L = M^{-1}S$$

- Eigenvectors v are M-orthogonal:
$$v_i^\top M v_j = \delta_{ij}$$
- First eigenvector v_1 is constant:
$$v_1 = \mathbf{1}$$

Symmetric Normalized Laplacian

$$L_{sym} = M^{-1/2} S M^{-1/2}$$

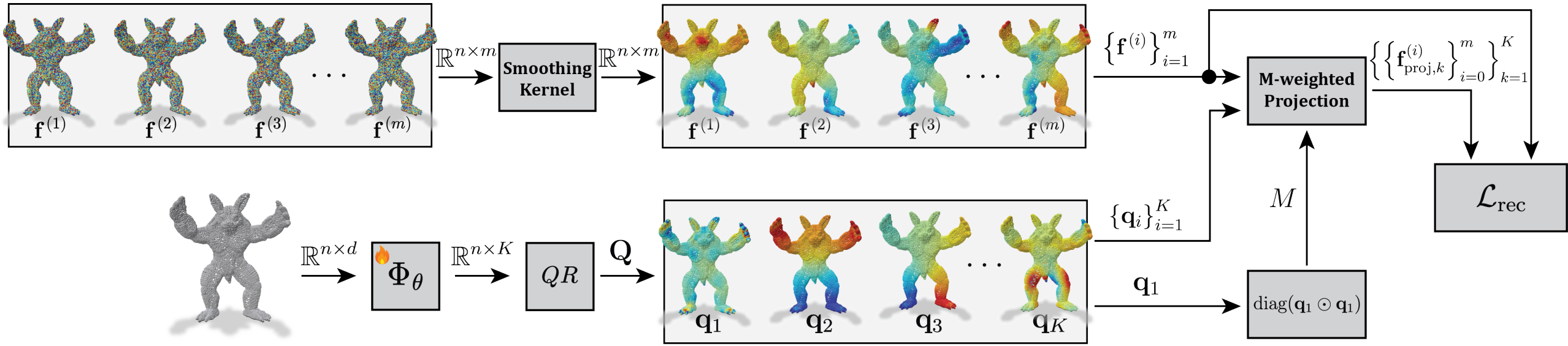
- Eigenvectors q are Euclidean-orthogonal:
$$q_i^\top q_j = \delta_{ij}$$
- First eigenvector encodes sampling density:
$$q_1 = M^{1/2}\mathbf{1}$$

The Relationship:

$$q = M^{1/2} v \;\leftrightarrow\; v = M^{-1/2} q$$

Same eigenvalues, related eigenvectors.

Our Pipeline



$$\mathcal{L}_{rec} = \frac{1}{mK}\sum_{i=1}^{m}\sum_{k=1}^{K}\left\| f^{(i)} - f_{proj,k}^{(i)} \right\|_2^2$$

Where:

- m **number of probe functions**
- K **number of eigenvectors**
- $f^{(i)}$ **the** i**-th probe function**
- $f_{proj,k}^{(i)}$ **projection of** $f^{(i)}$ **onto the first** k **basis vectors**

Why M-Weighted Projection?

The Brezis Theory

Uses standard Euclidean projection for $f_{proj,k}^{(i)}$, implicitly assumes **uniform sampling**.

The Problem in Practice

Point clouds are often **non-uniformly sampled**:

- Dense regions have many points
- Sparse regions have few points

Euclidean projection treats all points equally, ignoring the underlying geometry.

We use M-weighted projection to respect the manifold's sampling density.

Why M-Weighted Projection?

Project probe function f onto $Q_k = [q_1, \dots, q_k]$ such that the residual is M-orthogonal:

$$\left(f - f_{proj,k}\right)^T M q_i = 0 \quad \text{for } i = 1, \dots, k$$

By writing $f_{proj,k} = Q_k c^{(k)}$, we get a linear system of equations:

$$G^{(k)} c^{(k)} = b^{(k)}$$

Where:

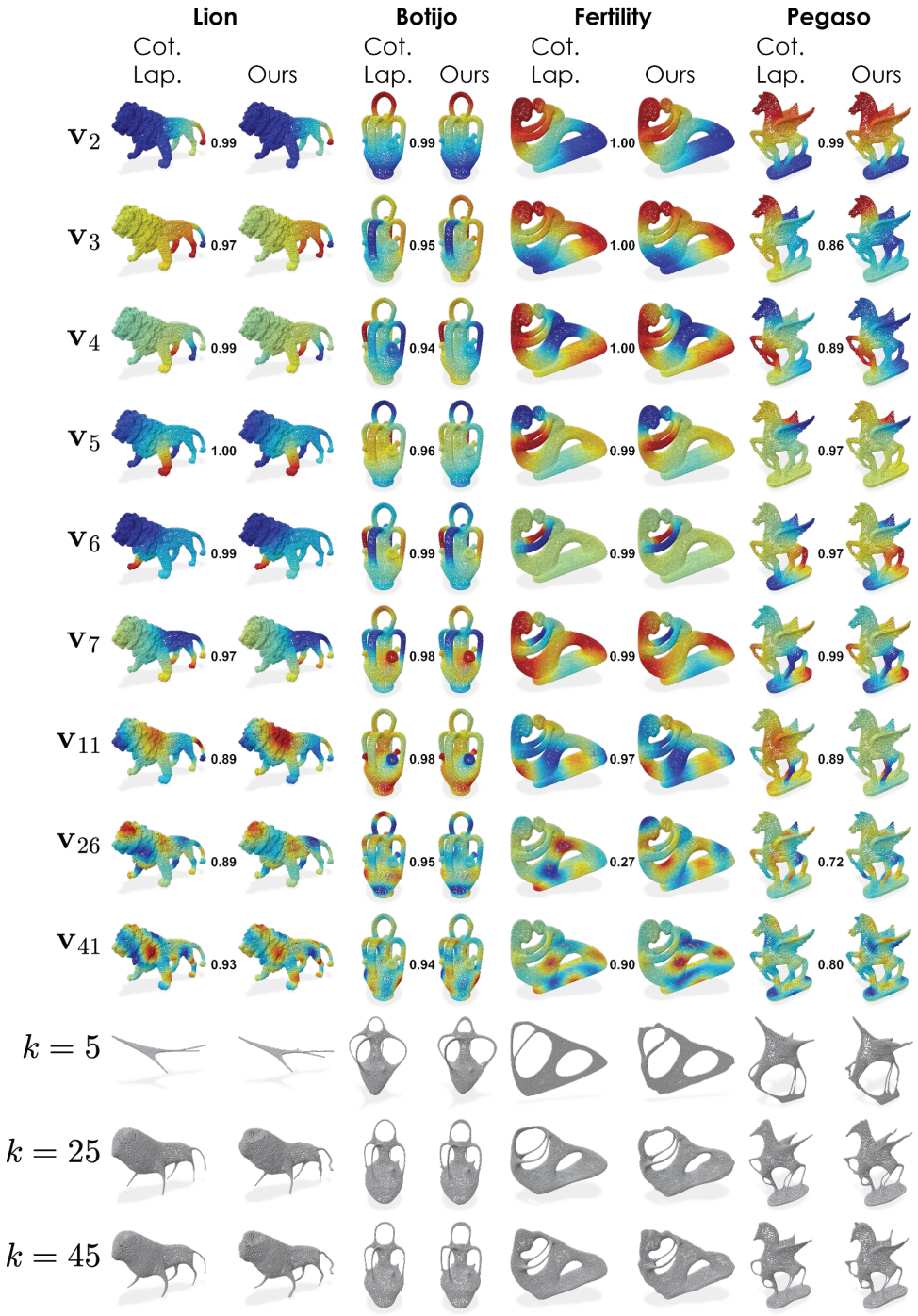
- $G^{(k)} = Q_k^\top M Q_k$ is the Gram matrix ($k \times k$)
- $b_i^{(k)} = q_i^\top M f$

Results – 3D Points Clouds

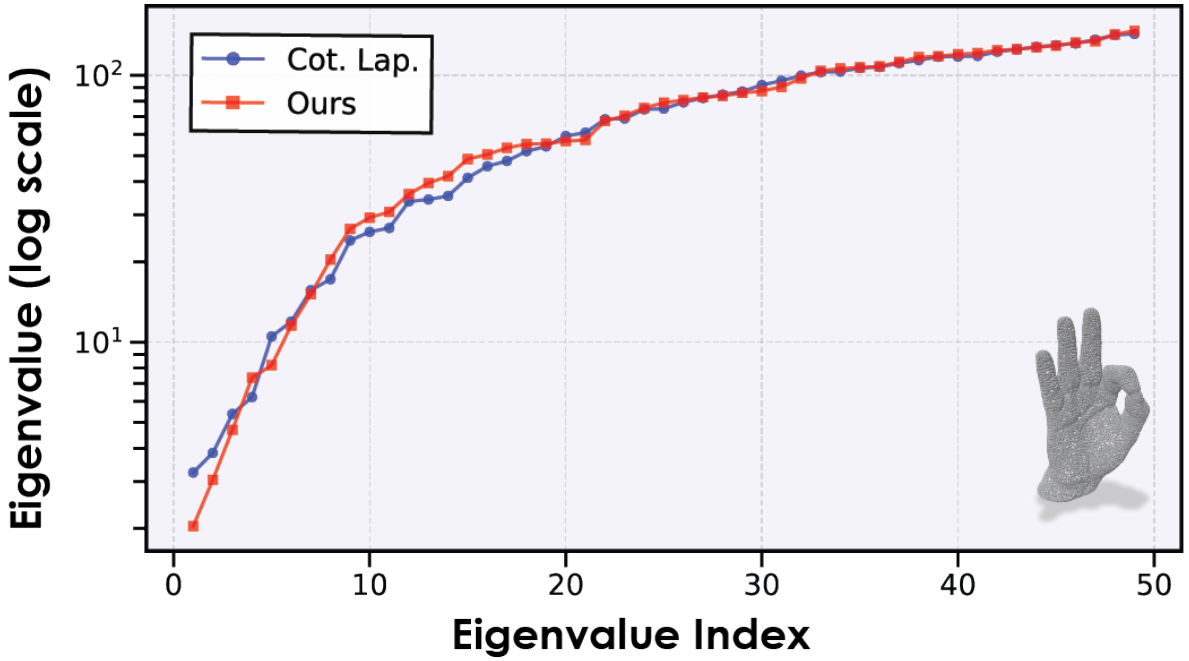
Overfit a Single Shape

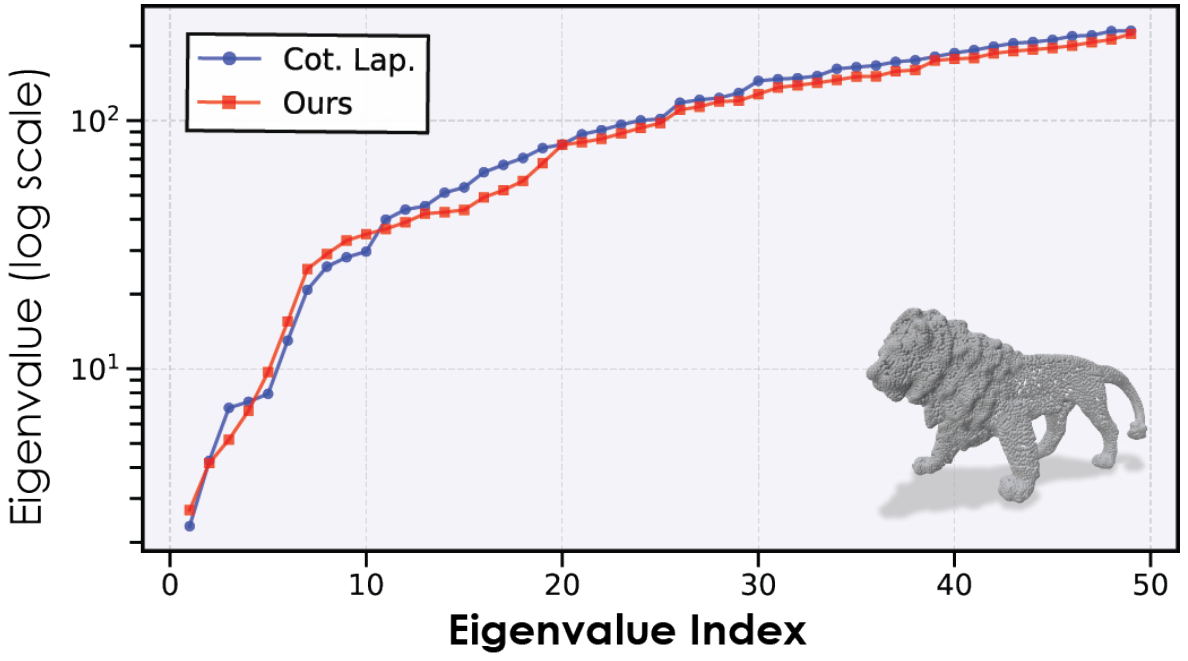
Table 1. Average cosine similarity between predicted and oracle eigenfunctions at different truncation levels k, and mean relative eigenvalue discrepancy (overfitting setting). More in Sec. C.

Shape	Image	$k \leq 10$	$k \leq 20$	$k \leq 50$	λ Discrepancy
Armadillo		0.968	0.967	0.773	0.200 ± 0.126
Bimba		0.964	0.945	0.822	0.093 ± 0.145
Botijo		0.972	0.955	0.813	0.153 ± 0.092
Elephant		0.979	0.866	0.687	0.105 ± 0.123
Fertility		0.874	0.866	0.720	0.083 ± 0.106
Kitten		0.993	0.988	0.981	0.088 ± 0.104
Laurent Hand		0.823	0.696	0.568	0.066 ± 0.078
Lion		0.951	0.908	0.822	0.067 ± 0.086
Pegaso		0.932	0.797	0.544	0.142 ± 0.140

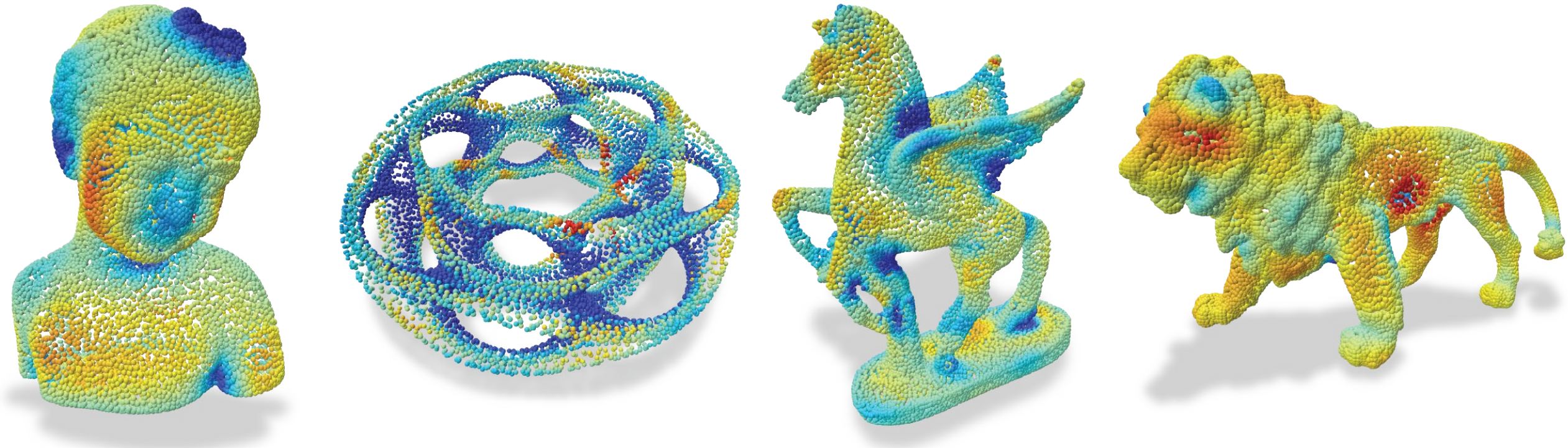


Overfit a Single Shape





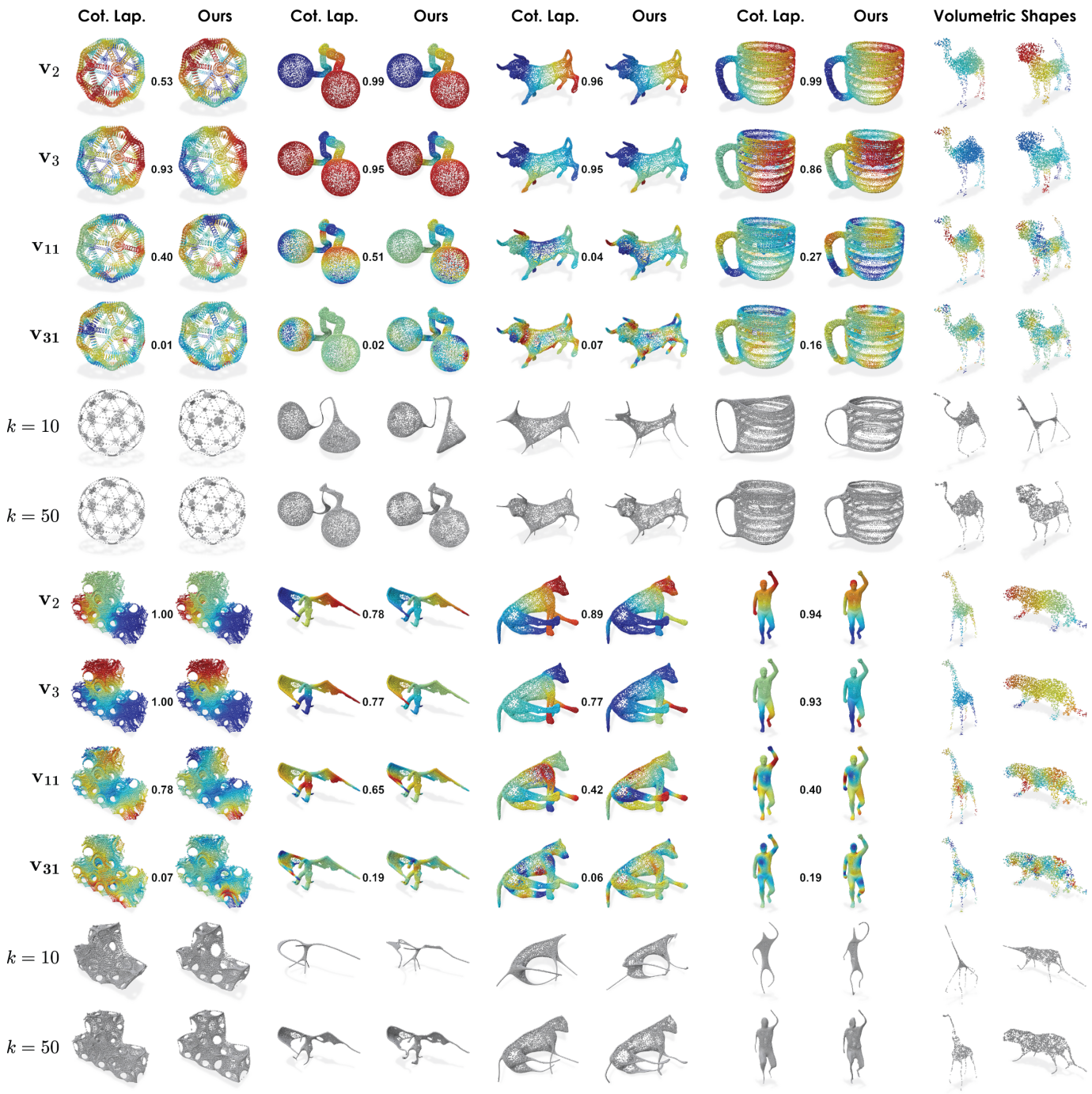
Overfit a Single Shape



Generalized Model

Trained on:

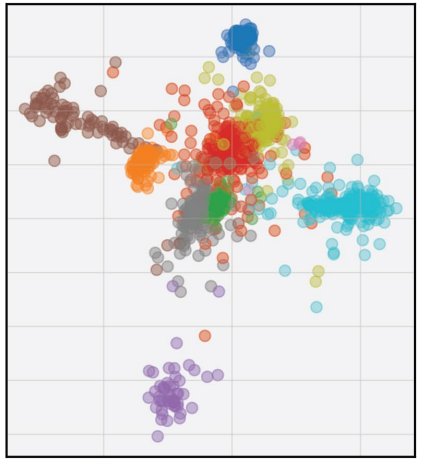
- **SURREAL** - for synthetic human bodies.
- **SHREC'21 Protein Domains** – for biological structures.
- **ABC** – for CAD models.
- **FAUST** – for real human scans.
- **DeformingThings4D** – for dynamic non-rigid surfaces.



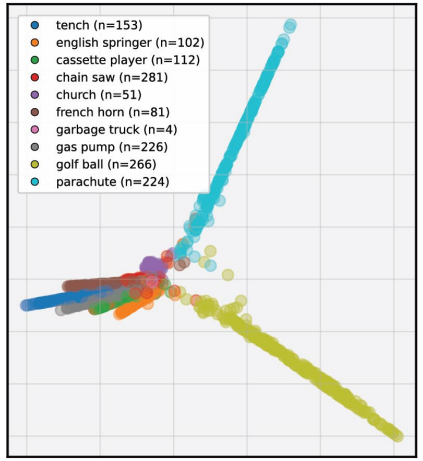
Results – Image Embeddings

DINOv2 Embeddings on Imagenette

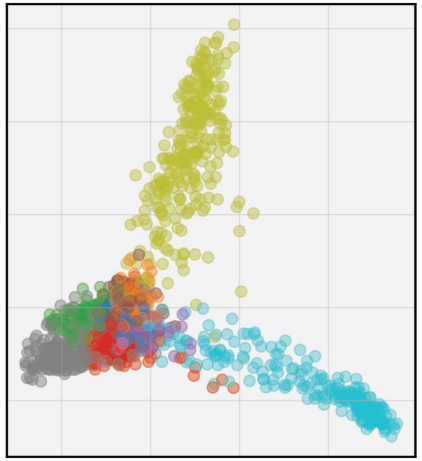
Opt. App. Eigenmaps (Ours)



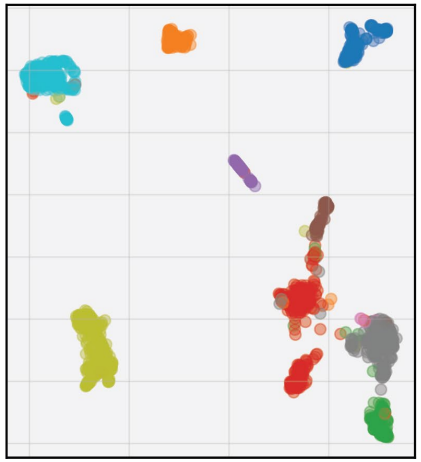
Laplacian Eigenmaps



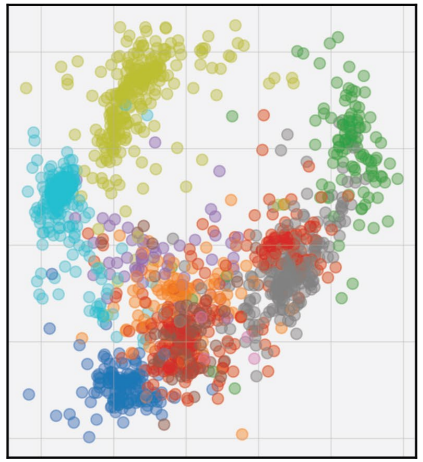
PCA



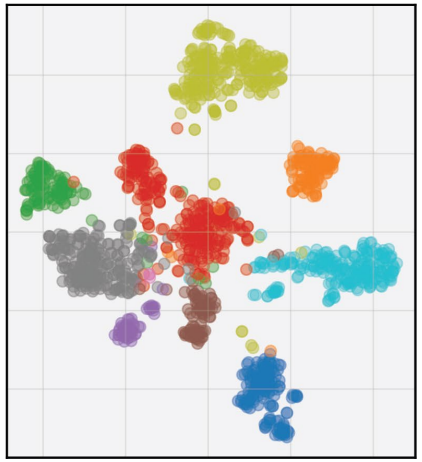
UMAP



Isomap

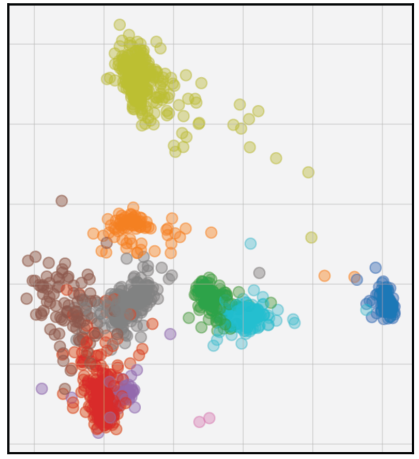


t-SNE

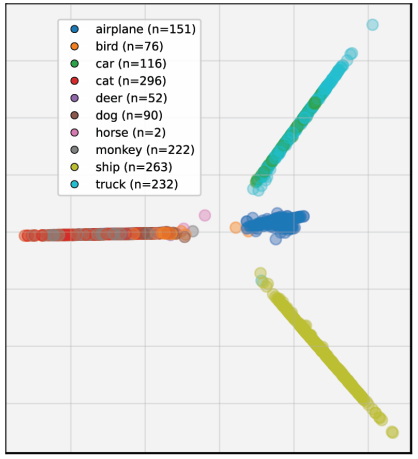


DINOv2 Embeddings on STL10

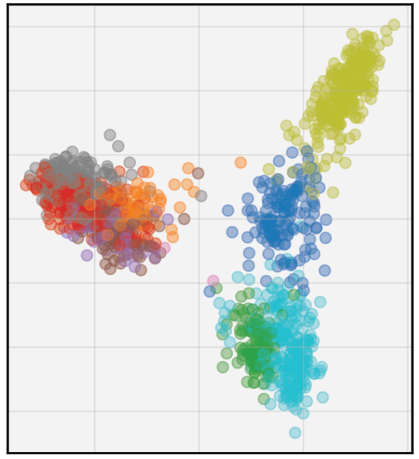
Opt. App. Eigenmaps (Ours)



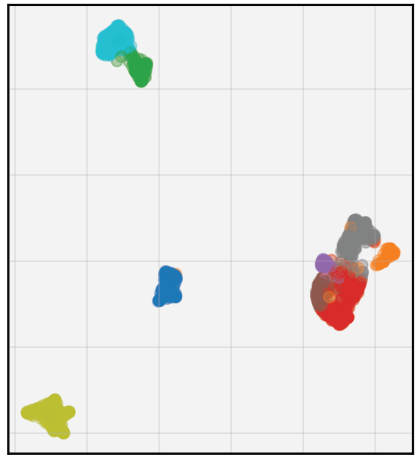
Laplacian Eigenmaps



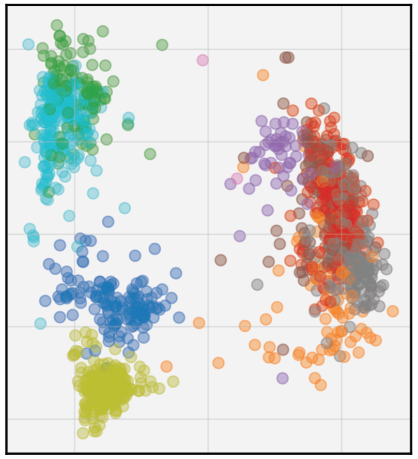
PCA



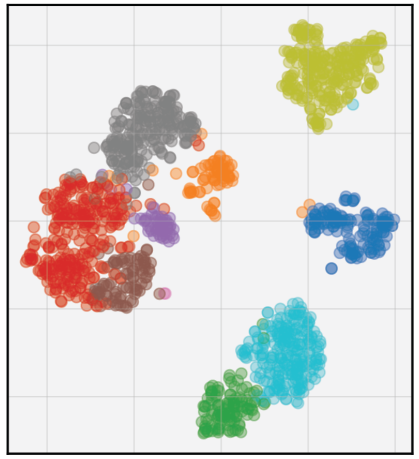
UMAP



Isomap

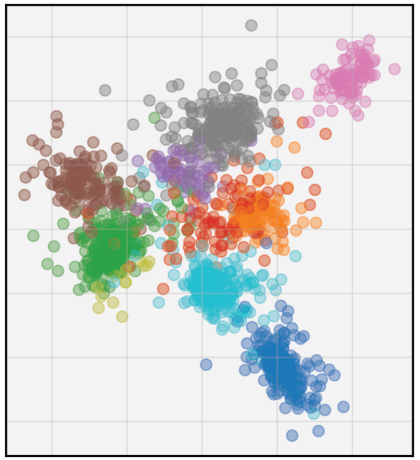


t-SNE

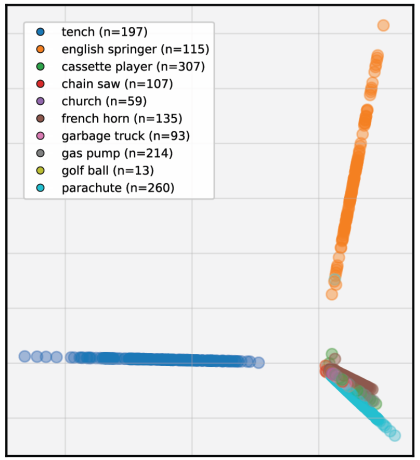


CLIP Embeddings on Imagenette

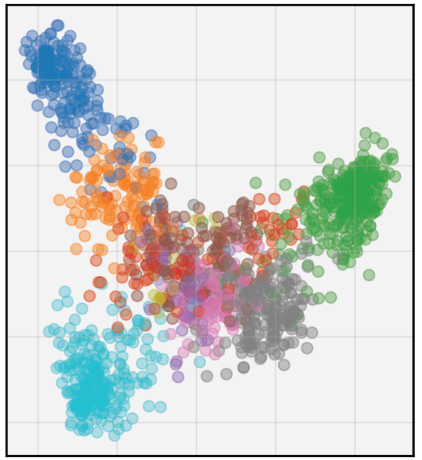
Opt. App. Eigenmaps (Ours)



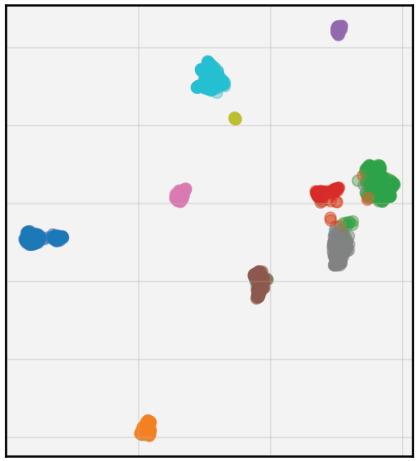
Laplacian Eigenmaps



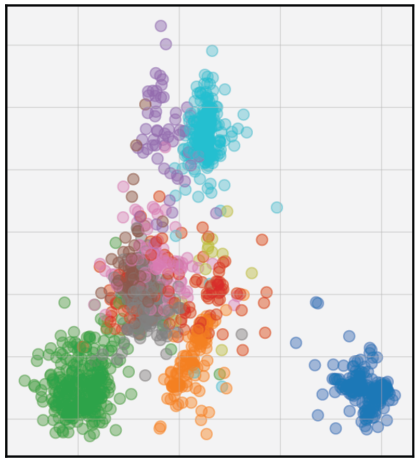
PCA



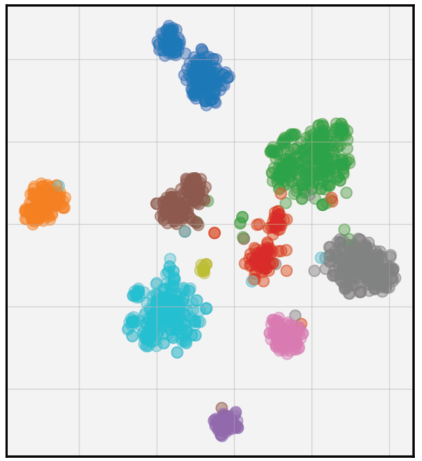
UMAP



Isomap

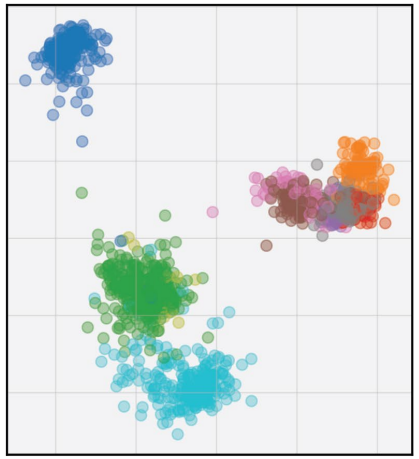


t-SNE

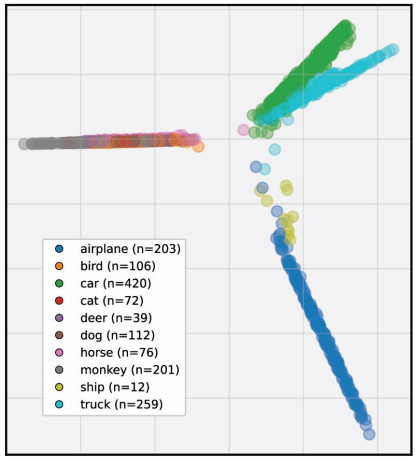


CLIP Embeddings on STL10

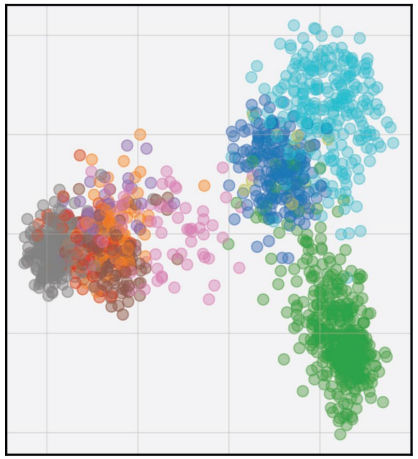
Opt. App. Eigenmaps (Ours)



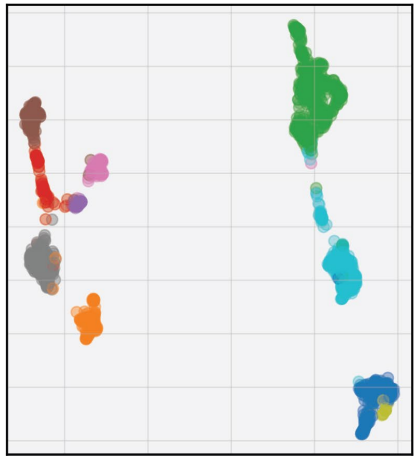
Laplacian Eigenmaps



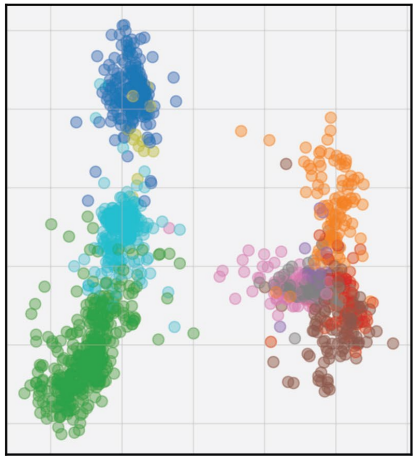
PCA



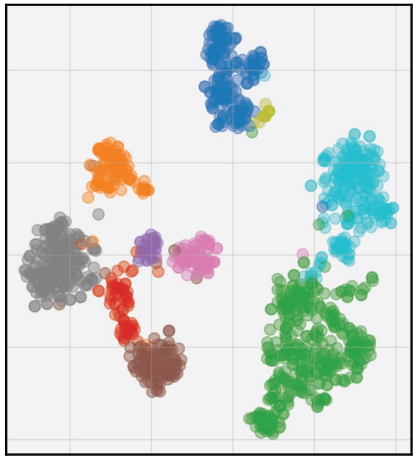
UMAP



Isomap

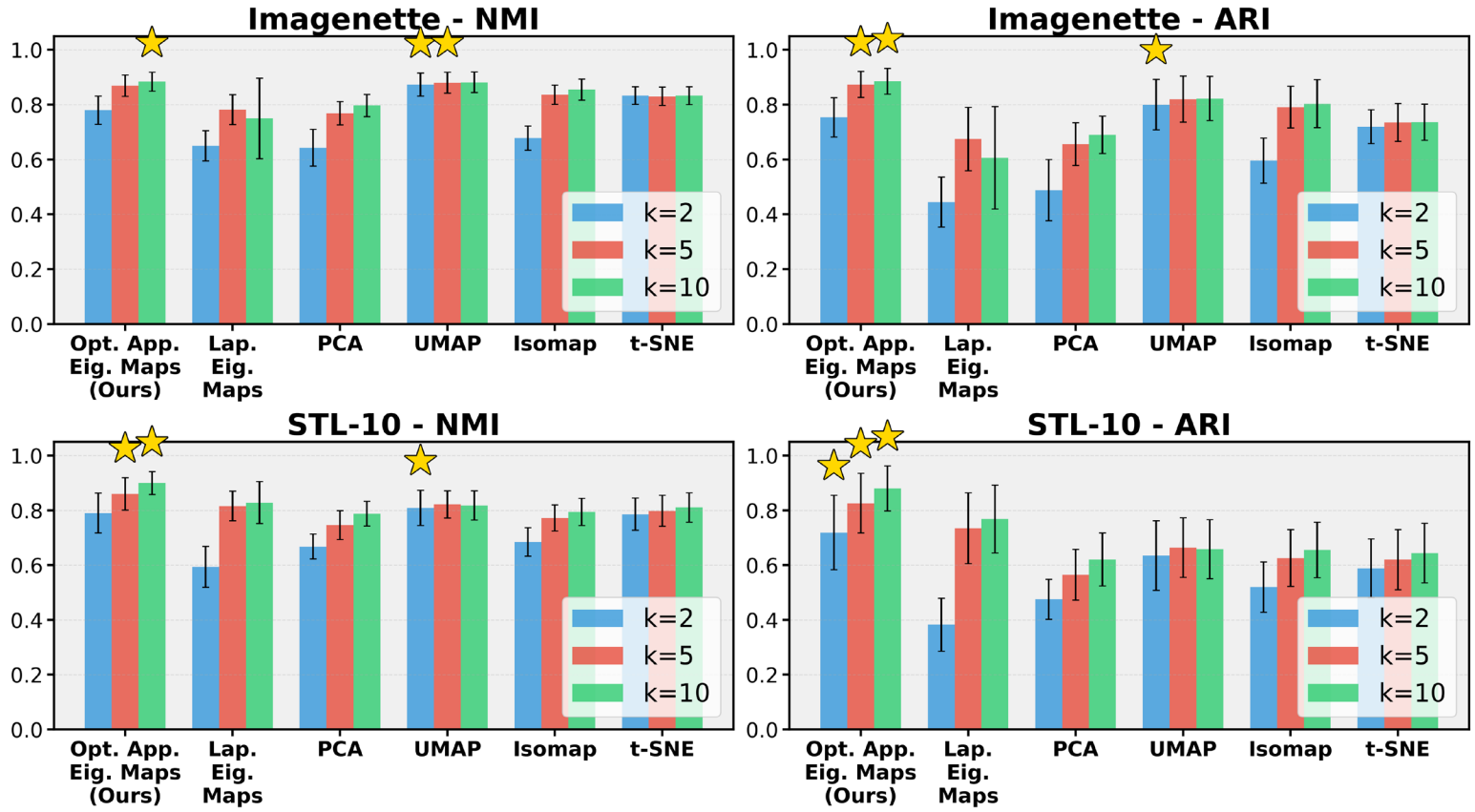


t-SNE



Clustring Quality

- Average clustering performance over 50 runs of manifold learning methods on DINOv2 features of random data subsets(1500 images).
- Higher is better.



NMI: If I tell you which cluster a point landed in, how much does that help you guess its true label? (1 = perfect hint, 0 = useless)

ARI: For every pair of points, did you correctly keep together the ones that should be together and separate the ones that should be apart? (1 = perfect, 0 = random chance)